Corteon / Barber Scheduler

Documento Atualizado de Lacunas do Sistema por Sprint

Comparação entre o backend atual e o planejamento oficial do produto.
Foco: o que ainda falta fechar, como fechar e em que ordem executar.

Status-base considerado neste documento

- Sprint 0 concluída
- Sprint 1 concluída
- Sprint 2 concluída
- Sprint 3 concluída
- Sprint 4 concluída
- Base atual com agenda, PIX, assinatura inicial, catálogo comercial, loja/carrinho e checkout público orquestrado já funcionando

Versão: diagnóstico operacional atualizado do backend

1. Contexto e objetivo do documento

Este documento consolida o que ainda falta no sistema após a conclusão das Sprints 0 a 4. Ele não reabre o que já está funcional sem necessidade. O foco é mostrar, por sprint, o que ainda precisa ser fechado para que o produto chegue ao objetivo final de ser a ferramenta central de operação da barbearia.

- Este documento assume uma evolução conservadora, sem refactor destrutivo.
- O backend continua sendo a fonte de verdade dos contratos antes de abrir as telas.
- A ordem proposta respeita dependência de domínio, impacto no produto e risco de retrabalho.
- Sempre que possível, a implementação deve ocorrer por bloco funcional utilizável, e não por limpeza estética.

2. O que já está sólido no backend

- multi-tenant por barbearia
- schema controlado por SQL em 000_init.sql
- timezone por barbearia
- agendamento público e privado
- conflito de horário protegido
- PIX seguro com idempotência
- payments separados para appointment e order
- jobs de expiração e no-show
- orders com itens e baixa de estoque no pagamento
- métricas persistidas de cliente
- política de pagamento por categoria
- plans, plan_services e subscriptions base
- catálogo comercial com serviço, produto e sugestão
- loja/carrinho públicos
- checkout público orquestrado

3. Débitos residuais das Sprints 0 a 4

- nomenclaturas herdadas ainda aparecem em partes do fluxo de serviço/agendamento
- há coexistência entre trilha legada de bookingpolicy e trilha viva de paymentconfig
- a stack de notification / email / ICS ainda precisa ser consolidada
- a camada HTTP ainda não está totalmente padronizada entre handlers, DTOs e mapeamento de erro
- a base de métricas de cliente existe, mas ainda não reflete toda a regra final de retenção e comportamento

4. Resumo de maturidade por objetivo de produto

Objetivo final	Definição	Leitura atual	Ação dominante
Agenda e operação base	90%	Forte	Manter e só lapidar
Inteligência de clientes / premium	60%	Parcial	Fechar regras de comportamento
Catálogo comercial	80%	Bem definido	Limpeza final de contratos
Assinatura / premium de negócio	70%	Funcional	Espalhar para operação e CRM

Loja / carrinho / jornada comercial	85%	Forte	Acabamentos de produto
Checkout público orquestrado	85%	Forte	Robustez e integração futura
Painel do dia	25%	Gap real	Criar query e card operacional
Fechamento operacional real	35%	Base pronta	Fazer realizado prevalecer
CRM do cliente	45%	Parcial	Consolidar contrato final
Dashboard geral	15%	Inicial	Criar query layer dedicada
Financeiro gerencial	10%	Muito inicial	Modelar realizado x expectativa
Comunicação / ticket / ICS	30%	Base parcial	Consolidar stack oficial
Relatório de impacto / ROI	5%	Quase não iniciado	Construir camada final de valor

Sprint 5 — Painel do Dia

Dar visão operacional real do dia e criar a primeira tela de trabalho diário do barbeiro.

Estado atual do backend para este bloco

- Hoje já existem listagem por data/mês, summary operacional simples, category, premium, assinatura ativa e sinais comerciais espalhados. O que ainda não existe é a leitura operacional unificada do dia.

O que ainda falta fechar

- query dedicada do painel do dia
- DTO único do card operacional
- leitura de item previsto e item pago antecipadamente por atendimento
- alertas operacionais no payload do card
- camada HTTP própria do painel do dia

Como implementar de forma conservadora

1. Criar um pacote de query dedicado, por exemplo internal/query/daypanel, sem reaproveitar listagem crua de appointments.
2. Modelar um DTO de card com: appointment_id, horário, cliente, categoria, premium, serviço agendado, status, item previsto, item pago antecipadamente, flags operacionais e contexto de assinatura.
3. Ler o dia sempre pela timezone da barbearia. A query deve receber barbershop_id, opcionalmente barber_id, e uma data local.
4. Montar a query em cima de appointments + clients + client_metrics + subscriptions ativas + appointment/order/payment necessários para os sinais comerciais.
5. Tratar 'item previsto' e 'item pago antecipadamente' como sinais separados. Item pago antecipadamente precisa aparecer de forma explícita no card.
6. Criar handler próprio, sem embutir essa lógica no appointment_handler já existente.
7. Fechar um endpoint administrativo específico, por exemplo GET /api/me/day-panel?date=YYYY-MM-DD&barber_id=... .
8. Testar cenários: cliente premium, cliente at_risk, atendimento com pagamento pendente, atendimento com item pago antes do dia e atendimento simples sem item previsto.

Partes do backend mais impactadas

- query layer nova
- DTO novo do card operacional
- handler operacional
- rota nova
- possíveis pequenos ajustes em repository de appointment/order/payment para leitura eficiente

Critério de pronto da sprint

- o front consegue renderizar o dia inteiro sem compor dados de vários endpoints
- cada card traz contexto suficiente para o barbeiro agir sem abrir outras telas
- o payload já nasce pronto para alimentar o fechamento operacional da sprint seguinte

O que não fazer nesta sprint

- não tentar fechar financeiro nesta sprint
- não misturar edição operacional no mesmo endpoint do painel
- não depender de cache para a primeira versão

Sprint 6 — Fechamento Operacional

Fazer o sistema refletir o que realmente aconteceu no atendimento, e não apenas o que foi agendado.

Estado atual do backend para este bloco

- O backend já possui appointment_closure, transação na conclusão, valor final, nota operacional, cobertura de assinatura e confirmação de cobrança normal. Isso é base, mas ainda não é fechamento operacional real.

O que ainda falta fechar

- troca de serviço realizado
- separação completa entre planejado e realizado
- venda adicional no fechamento
- remoção ou substituição de item previsto
- forma de pagamento real e momento do pagamento
- integração do realizado com métricas e financeiro futuro

Como implementar de forma conservadora

1. Criar um contrato de fechamento operacional próprio, em vez de continuar expandindo apenas o payload atual de complete appointment.
2. Separar explicitamente no fechamento: serviço agendado, serviço realizado, valor de referência, valor final, itens previstos, itens vendidos e itens removidos.
3. Permitir que venda adicional gere order normal dentro do fechamento, para manter estoque e financeiro coerentes com o restante do sistema.
4. Modelar forma de pagamento real e momento do pagamento em um bloco operacional separado do payment técnico de plataforma.
5. Consumir assinatura com base no serviço final realizado, não no serviço originalmente agendado.
6. Expandir auditoria para registrar: fechamento, troca de serviço, ajuste de valor, venda adicional e confirmação de pagamento.
7. Executar tudo em transação, porque o fechamento passa a ser o ponto mais sensível do atendimento.
8. Manter um endpoint específico de fechamento operacional, em vez de acoplar tudo a create/update genéricos.

Partes do backend mais impactadas

- novo DTO de fechamento
- use case de fechamento operacional
- ajustes em appointment_closure
- integração com order
- integração com subscription consume
- auditoria ampliada

Critério de pronto da sprint

- o barbeiro consegue registrar o que realmente aconteceu
- o sistema passa a ter diferença explícita entre planejado e realizado
- o valor final, o item vendido e o serviço realizado ficam persistidos de forma consistente

O que não fazer nesta sprint

- não abrir ajuste pós-fechamento invisível
- não usar o payment técnico como se ele fosse pagamento operacional completo
- não deixar venda adicional fora de order

Sprint 7 — CRM do Cliente + Ajuste Pós-Fechamento

Consolidar leitura útil do cliente e permitir correções controladas após o fechamento.

Estado atual do backend para este bloco

- Já existe base de client_history, category, premium e métricas persistidas. Também existe audit base e appointment_closure. Ainda faltam contrato final do CRM e ajuste posterior rastreável.

O que ainda falta fechar

- contrato final do CRM individual
- policy aplicada no CRM
- flags finais (premium, reliable, attention, at_risk)
- ajuste pós-fechamento por janela limitada
- rastreabilidade entre fechamento original e correções

Como implementar de forma conservadora

1. Criar um endpoint oficial de CRM individual, separado de listagem simples de clientes.
2. Fazer o CRM consolidar: identidade do cliente, category, premium, totais, attendance, cancelamentos, no-show, reagendamentos, última visita, assinatura ativa e policy aplicada.
3. Revisar o classificador de métricas para refletir as regras finais de retenção, 30 dias, janela de 90 dias, no-show como pior sinal e penalidades tardias.
4. Criar um fluxo de ajuste posterior por até 7 dias, sempre registrando o delta como ajuste e nunca como edição invisível do fechamento original.
5. Ampliar auditoria para diferenciar fechamento original e correção posterior.
6. Manter o ajuste pós-fechamento em endpoints próprios, não em update genérico de appointment.

Partes do backend mais impactadas

- query/DTO final de CRM
- revisão de metrics
- handlers do CRM
- novo fluxo de ajuste pós-fechamento
- auditoria adicional

Critério de pronto da sprint

- o barbeiro consegue entender rapidamente o comportamento do cliente
- qualquer correção posterior fica rastreável
- o CRM deixa de ser histórico parcial e vira contrato final de leitura do cliente

O que não fazer nesta sprint

- não reabrir categoria premium
- não misturar ajuste posterior com edição livre
- não inflar o CRM com dados financeiros ainda não consolidados

Sprint 8 — Dashboard Geral

Entregar visão macro da produção e da venda em períodos úteis para gestão.

Estado atual do backend para este bloco

- Existem blocos de base: appointments, orders, subscriptions, payments, metrics e operational summary. Ainda não existe query geral consolidada nem contrato final de dashboard.

O que ainda falta fechar

- query consolidada por período
- contrato único do dashboard
- recortes de cliente
- receita por origem
- rankings de serviço e produto

Como implementar de forma conservadora

1. Criar uma query layer própria para dashboard, sem tentar montar esse payload a partir de endpoints já existentes.
2. Fechar primeiro a versão dia/semana/mês com filtros simples e previsíveis.
3. Tomar como métrica principal total de atendimentos concluídos e usar o restante como complementos.
4. Separar receita por origem: serviços, produtos e assinatura.
5. Criar rankings por faturamento, não apenas por volume.
6. Garantir DTO leve para mobile, com cards agregados e listas curtas de ranking.
7. Só considerar pré-agregação se a query direta realmente provar ser insuficiente.

Partes do backend mais impactadas

- query layer financeira e operacional
- DTO do dashboard
- handler do dashboard
- rotas administrativas novas

Critério de pronto da sprint

- o dashboard retorna visão macro do período sem depender de composição no front
- a barbearia consegue enxergar produção, venda e receita em uma única leitura

O que não fazer nesta sprint

- não começar por gráficos complexos
- não misturar CRM e painel do dia com dashboard geral
- não usar cache como solução inicial

Sprint 9 — Financeiro: Realizado, Expectativa, Presumido e Perdas

Transformar o backend de financeiro técnico em leitura financeira de negócio.

Estado atual do backend para este bloco

- Hoje existem payments, orders, appointment_closures e subscriptions, mas ainda não existe separação formal entre realizado, expectativa e presumido.

O que ainda falta fechar

- faturamento realizado por período
- expectativa baseada em agenda futura e itens previstos
- presumido baseado no planejado ainda não fechado
- fechamento semanal e mensal
- perdas por no-show, cancelamento e item previsto não vendido

Como implementar de forma conservadora

1. Criar contratos financeiros separados do dashboard operacional, mesmo que o dashboard depois consuma esses números.
2. Definir realizado como aquilo que já foi de fato fechado ou faturado conforme a regra oficial do período.
3. Definir expectativa com base na agenda futura e nos itens previstos ainda abertos no período.
4. Definir presumido como o valor planejado que ainda não virou realizado por fechamento manual.
5. Modelar perdas explicitamente: no-show, cancelamento tardio, produto previsto não vendido e atendimento não concluído.
6. Criar fechamento semanal oficial no domingo e fechamento mensal consolidado.
7. Se necessário, preparar snapshots ou pré-agregações, mas só depois de fechar a semântica do financeiro.

Partes do backend mais impactadas

- query layer financeira
- possíveis tabelas de fechamento/snapshot
- DTOs financeiros
- rotas administrativas
- integração com appointment_closure e order

Critério de pronto da sprint

- o sistema passa a explicar o quanto foi realizado, o quanto ainda era esperado e o que foi perdido
- a leitura deixa de ser só soma de eventos técnicos e vira visão de negócio

O que não fazer nesta sprint

- não começar por relatório bonito sem semântica sólida
- não contar previsão como realizado
- não esconder perdas dentro de métricas genéricas

Sprint 10 — Comunicação, ICS e Ticket do Agendamento

Melhorar a experiência do cliente sem mexer no core do domínio.

Estado atual do backend para este bloco

- Já existe base de email, notificação e ICS parcial. O que falta é consolidar stack oficial, ligar aos eventos certos e criar ticket/link único do agendamento.

O que ainda falta fechar

- ticket público de gerenciamento do agendamento
- cancelamento/reagendamento pelo cliente com regras oficiais
- contagem de reagendamentos por agendamento
- penalidade negativa após 24h
- envio oficial de email/ICS nos eventos certos

Como implementar de forma conservadora

1. Modelar um token ou ticket único vinculado ao appointment, com acesso restrito apenas ao próprio agendamento.
2. Criar endpoints públicos de consulta, cancelamento e reagendamento via ticket, respeitando as regras temporais da barbearia.
3. Contar reagendamentos no próprio agendamento para suportar limite máximo configurável futuramente.
4. Aplicar métrica negativa apenas quando o evento ocorrer na janela tardia definida.
5. Consolidar uma única stack oficial de notificação, em vez de manter caminhos paralelos para email e ICS.
6. Disparar notificações de forma assíncrona, sem bloquear o fluxo crítico do backend.

Partes do backend mais impactadas

- modelo/serviço de ticket público
- handlers públicos específicos
- integração oficial de notification
- consolidação de ICS

Critério de pronto da sprint

- o cliente consegue gerenciar o próprio agendamento com segurança
- os emails e arquivos ICS deixam de ser infraestrutura solta e passam a ser parte oficial do produto

O que não fazer nesta sprint

- não acoplar notificação síncrona ao fluxo crítico
- não expor acesso público sem token seguro
- não abrir reagendamento público sem contagem e regra temporal

Sprint 11 — Relatório de Impacto / ROI

Criar a camada final de valor percebido do sistema.

Estado atual do backend para este bloco

- Quase não iniciado como produto. Existem apenas os dados-base que vão alimentar esse relatório no futuro.

O que ainda falta fechar

- relatório consolidado de faturamento gerado
- crescimento
- retenção
- perdas
- ROI
- uso do sistema
- ganhos indiretos

Como implementar de forma conservadora

1. Consumir dashboard, financeiro, métricas e retenção já consolidados nas sprints anteriores.
2. Fechar um contrato leve e legível, orientado a justificar valor do produto para o barbeiro.
3. Mostrar evolução temporal, não apenas foto estática do período.
4. Separar claramente números observados de estimativas derivadas.
5. Tratar esse relatório como camada de produto e venda, não como relatório técnico.

Partes do backend mais impactadas

- query layer final
- DTO de relatório de impacto
- integração com financeiro e métricas
- rota administrativa específica

Critério de pronto da sprint

- o sistema passa a justificar mensalidade com visão consolidada de resultado
- o relatório consegue mostrar valor gerado e não só atividade do software

O que não fazer nesta sprint

- não começar esse bloco antes do financeiro e dashboard estarem fechados
- não misturar métricas derivadas com números realizados sem sinalização

12. Regras transversais para todas as próximas sprints

- Manter schema evoluindo diretamente em 000_init.sql, conforme a regra atual do projeto.
- Toda ação sensível nova deve disparar auditoria: fechamento, ajuste, troca de serviço, venda adicional, alteração de plano e alteração de regras da barbearia.
- Não abrir tela nova em cima de endpoint provisório; primeiro fecha contrato do backend, depois abre front.
- Não usar cache como dependência crítica; primeiro fecha modelagem e query correta.
- Preservar a separação entre appointment, order, payment e subscription; experiência pode ser unificada, domínio não precisa ser colapsado.
- Não fazer refactor destrutivo por estética. Só limpar o que bloquear entendimento, manutenção ou contrato.

13. Ordem prática recomendada a partir de agora

1. Sprint 5 — Painel do Dia
2. Sprint 6 — Fechamento Operacional
3. Sprint 7 — CRM do Cliente + Ajuste Pós-Fechamento
4. Sprint 8 — Dashboard Geral
5. Sprint 9 — Financeiro
6. Sprint 10 — Comunicação / ICS / Ticket
7. Sprint 11 — Relatório de Impacto

Leitura final

- O backend já está forte no motor transacional e comercial básico.
- O próximo salto de valor do produto está na operação do dia, no realizado vs planejado, no CRM e no financeiro.
- As próximas sprints não precisam reinventar a base; precisam transformar a base já pronta em contratos operacionais maduros.